

Chakravyuh

Intercept the scam workflow, not the transaction.

Track 02 · AI-Driven Scam Pattern Recognition in UPI & banking

A stateless kill-chain decision engine that scores UPI payment sessions on 20 device + bank + network signals, catches scam workflows early, and gives users a specific, human reason -- with a signed audit receipt.

Team

Mathews V Manoj · Muthoot Institute of Technology and Science, KTU

Undergraduate, Electronics & Communication Engineering · 24ec357@mgits.ac.in

The problem, in one screen

Real-time UPI + AI-generated pretexts have collapsed the time users have to notice a scam.

₹22,845 cr

lost to cyber fraud in India, 2024

36.37 lakh complaints (NCRP + CFCFRMS)

+206%

year-on-year jump

vs. ₹7,465 cr in 2023

₹981 cr

UPI fraud in FY25

12.64 lakh incidents (RBI Annual Report 24-25)

Why yesterday's rules stop catching this

AI-cloned voices, deepfaked video calls, task-scam bait credits, and rented mule accounts flip the strong single-signal rules banks use today into 40%+ false-positive noise. The fraudster no longer sends money -- the victim does.

What victims actually experience

- A caller they trust (police, delivery, HR, family) sets the frame in <60 s.
- A second app or overlay quietly gets consent to the screen.
- A large, first-time transfer flows to a personal VPA opened in the last month.
- The 'success' screen arrives before the family notices.

Chakravyuh -- one sentence

A stateless decision engine that reads a UPI session as a 4-stage kill chain and interrupts only when a *workflow* forms.

The kill chain we watch

1. Contact

unknown call, video call, scam
SMS lure

2. Control

screen-share, sideloaded APK,
overlay, OTP-in-call

3. Extraction

first-time payee, typed VPA,
unusual amount, staircase, FD
broken

4. Cash-out

young account, mule fan-in,
I4C registry hit

Three interventions, one budget

- Tier 1 -- silent watch-list entry. No user friction. Feeds the bank's model.
- Tier 2 -- pre-payment interrupt with a specific reason and two safety questions. Only fires when ≥ 2 kill-chain stages have credible evidence.
- Tier 3 -- 30-minute cooling-off hold for high-value + multi-stage sessions. User can call the bank; agent sees the same explanation.
- Alert budget is set by the bank (default: 3 interruptions per 1,000 sessions) and the engine self-calibrates thresholds to hit it -- no alert fatigue.

Before and after -- the actual user beat

Same victim, same scammer, two futures. This is the differentiation reviewers can feel.

Today · no Chakravyuh

1. Fake 'CBI officer' video-calls Mrs. R. Says her name is on a money-laundering case.
2. Tells her to install AnyDesk 'for verification'.
3. Asks her to break the FD 'for asset freeze' and pay to a personal VPA.
4. Bank rule: high amount + new payee -> soft warning. She reads it aloud on the call. She proceeds.
5. ₹4,20,000 lost. Complaint at NCRP the next morning. Recovery: rare.

With Chakravyuh

- 1-3. Same first three steps. On-device SDK sees: unknown video call, screen-share, FD broken in the last 24h.
4. When she presses Pay, engine sees 3 kill-chain stages fire. Tier-3 cooling-off hold kicks in.
5. The screen shows: 'Real CBI / police never demand payment on a call. This account was opened 8 days ago.'
6. She calls her son. Bank agent sees the same signed decision receipt. Payment released or reported.
7. If she still wants to pay, she can -- after 30 minutes and a video-KYC step at the branch.

Core detection & decision logic

Additive, non-negative evidence weights on 20 signals -- learned, capped per stage, gated by chain.

How a session gets its score

$$\text{score(s)} = \text{prior} + \sum_{\text{stage}} \min(\text{cap_stage}, \sum w_k \cdot x_k)$$

- x_k are 20 binary signals from device / bank / network (Slide 6).
- w_k are non-negative weights learned by L2-regularised logistic regression on labelled sessions. Absence of a signal never raises OR lowers the score -- fraudsters can suppress signals.
- Each stage is capped so no single stage can breach a tier alone.
- Tier 2 requires ≥ 2 active stages. Tier 3 requires ≥ 3 stages, or 2 stages + I4C suspect-registry hit on the payee.
- Prior is shifted from training prevalence to real prevalence before calibration -- thresholds are hit on a validation world.

Top learned weights (log-odds)

payee mule high	+5.74
collect from p2p	+4.66
payee registry hit	+3.43
bait credit 7d	+3.00
first time payee	+2.73
balance drain	+2.41
sideload 24h	+2.32
vpa typed	+2.08
fd broken 24h	+1.93
amount high	+1.71
call long	+1.52
sms scam flag	+1.50

Weights are non-negative; correlated signals share weight (no double counting).

What the engine reads (and what it never sees)

All 20 signals live in one catalogue. Raw content stays on-device.

Contact & pretext

- **call unknown active**
You are on a call with a number not in your contacts while paying
- **call long**
That call has lasted more than 20 minutes
- **video call**
A video/VoIP call is active (common in 'digital arrest' scams)
- **sms scam flag**
A message received in the last 24h matches a known scam lure (KYC, parcel, electricity)
- **bait credit 7d**
Small 'profit' credits from unknown senders in the last 7 days

Control & isolation

- **remote access**
A screen-sharing / remote-control app is running
- **sideload 24h**
An app was installed from outside the Play Store in the last 24h
- **accessibility overlay**
A non-system app can read your screen and draw over the payment app
- **otp read in call**
An OTP was opened while the unknown call was active

Money extraction

- **first time payee**
You have never paid this account before
- **amount high**
Amount is far above your usual payments
- **amount very high**
Amount is extremely high for you (top 0.1% of your history)
- **balance drain**
This payment empties more than 70% of your balance
- **staircase**
Several payments to new accounts in the last 60 minutes
- **fd broken 24h**
A fixed deposit was broken / loan taken in the last 24h
- **vpa typed**
Payee was typed or pasted instead of scanned or picked from contacts
- **collect from p2p**
This is a 'collect' request from an individual (entering PIN SENDS money)

Mule / cash-out

- **payee mule high**
The receiving account behaves like a mule account (many unrelated senders, money moved out fast)
- **payee new account**
The receiving account was opened less than 30 days ago
- **payee registry hit**
The receiving account is on the I4C / bank suspect registry

Never leaves the device / never stored

Call audio, message text, screen contents, contact list, app inventory, OTP values. The engine sees only 20 booleans + amount + payee VPA hash. Every decision generates a signed receipt so a hold can be audited or appealed.

Evidence -- does it work on synthetic data?

3 unseen synthetic worlds · 60,000 legit + 2,400 scam sessions per world · alert budget 3/1,000

92%

of scam sessions caught (tier ≥ 2)

vs. 49% for the 'new payee + high amount' rule

3.6 / 1,000

interruptions to legit users

vs. 9.2 for the rule -- and we tell users why

96%

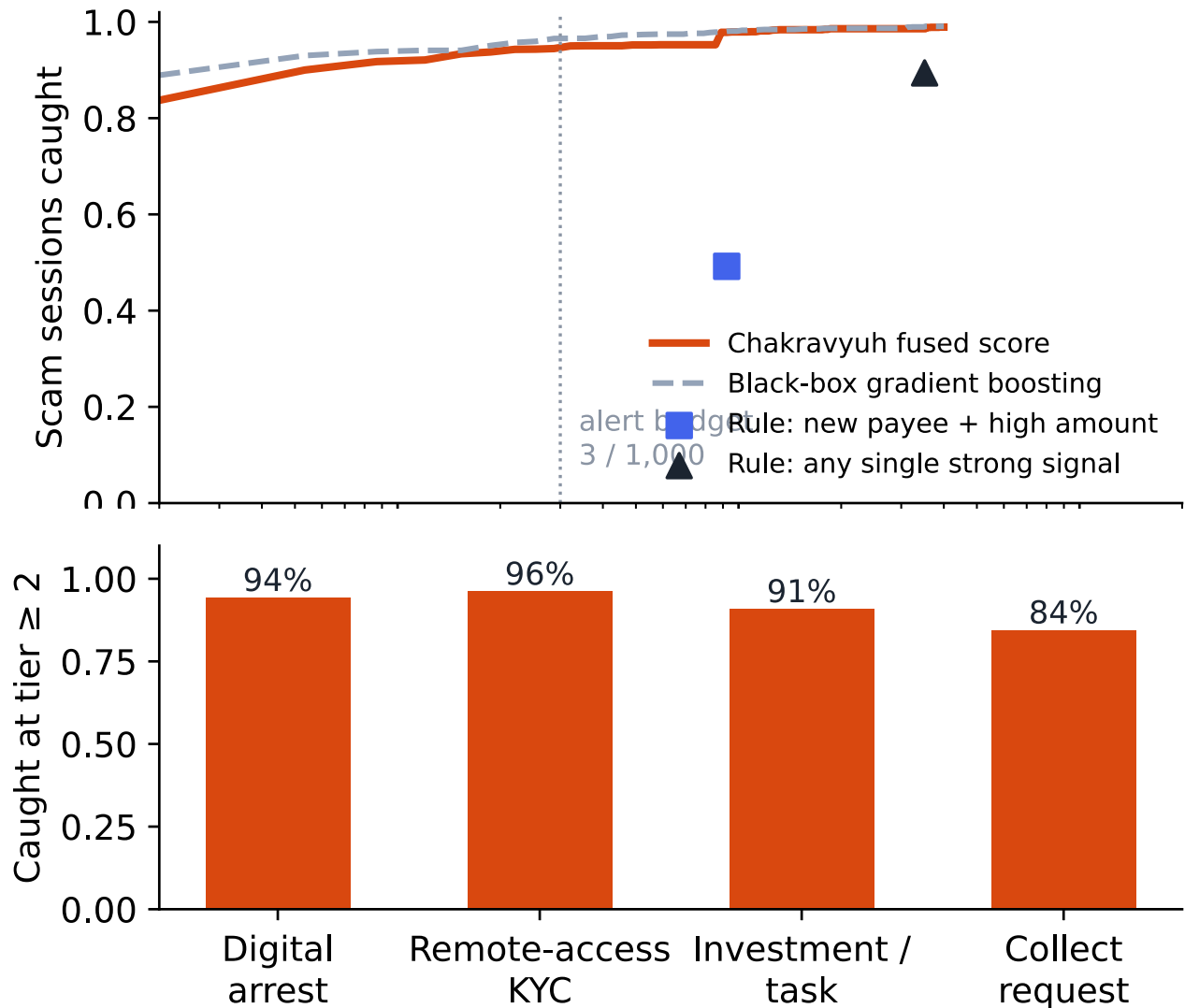
of scam ₹ intercepted

value-weighted, not just count

88%

recall on adapted fraudsters

suppressed call/screen-share signals; drop only 5 pp



Black-box gradient boosting at the same budget: 97% recall -- we trade ~4 percentage points for stage-gated explanations & auditability.

Trust -- what if we are wrong, and how we resist misuse

Every safeguard visible in the deck must appear in the architecture too (Round-1 rubric).

What data do you need?

20 booleans, amount, payee hash. No audio, no message text, no screen frames, no contact upload.

What happens when we are wrong (FP)?

Tier 2 is a friction step, never a block: 2 questions, 15-second delay, then user can proceed. Tier 3 hold is releasable by a human bank agent within 30 minutes.

What happens when we are wrong (FN)?

Post-hoc: every tier-1 receipt goes to the bank's fraud queue. Recovered ₹ loops into the mule graph.

Who can act on the result?

The user sees a message and can pause. Only a bank agent (dual auth) can release a tier-3 hold or add an account to the suspect registry.

How will users understand the result?

A specific playbook name and one sentence -- never just a score. Available in EN / HI / regional.

How will it resist misuse?

Signed receipts (HMAC) so a hold is auditable; on-device inference for privacy-sensitive signals; rate-limited registry writes; no direct block on a single signal; adversarial-adaptation coverage (88% recall on suppressed-signal sessions).

Adoption, integration, and scale

Pilot with one PSU or private bank; plug the same engine into UPI PSP + bank SDK later.

Where it sits

- Bank-side: stateless FastAPI in the bank's fraud-tech VPC.
- Client-side: 200-line SDK inside the bank app.
- Rail: pre-approved hook on the UPI PSP for tier-3 holds.

Who owns what

- Bank fraud-ops team: owns thresholds, alert budget, tier-3 releases.
- Bank data team: owns the mule graph (nightly refresh, I4C registry ingest).
- Product: owns copy per playbook and per language.

Rollout

- Weeks 0-6: shadow mode. Emit receipts, never intervene.
- Weeks 6-10: tier-1 + tier-2 on senior segment and >₹50k P2P.
- Weeks 10+: tier-3 hold with agent-desk workflow; add second bank.

Operational metrics we will report weekly

25 μs engine latency per session (in-process)	< 40 ms p99 SDK round-trip in target design	3.0 / 1k interruptions to legit users -- SLO	< 24 h median time from receipt to human review
---	--	--	--

48-hour finale plan -- what we can demonstrate live

Every deliverable maps to a rubric line and to a file already in this repo.

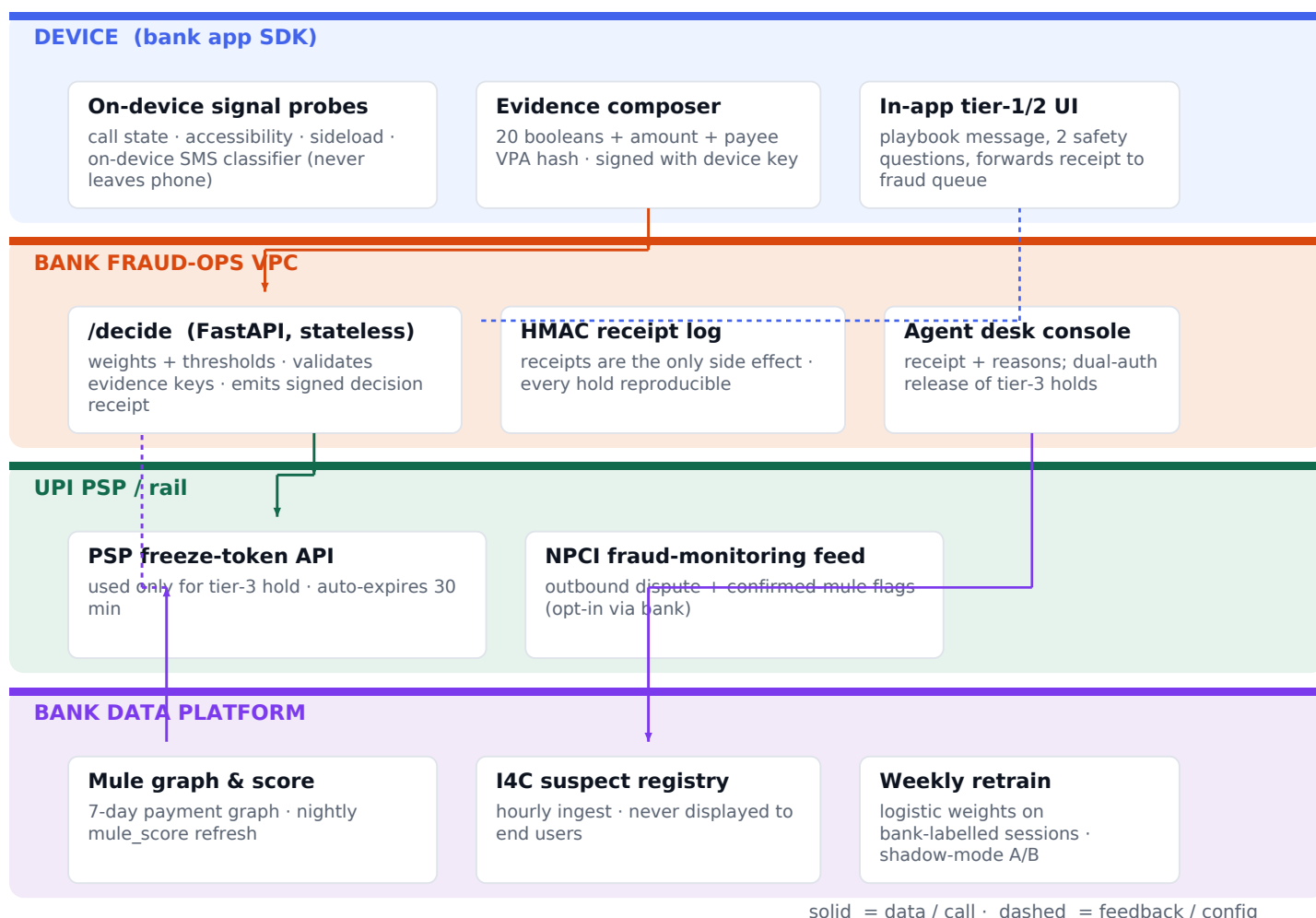
- H+0-6** Freeze the signal catalogue, receipt schema. Bank-side FastAPI wired against synthetic replay.
- H+6-18** Android SDK stub (Kotlin) reading real device signals (call state, accessibility, sideload). Replaces the synthetic device signals in the demo.
- H+18-28** Live playback: 100 recorded 'sessions' (synthetic) into the SDK -> engine -> bank agent dashboard. Live tier-1/2/3 decisions with signed receipts.
- H+28-40** Attack team: adapt 3 playbooks against the engine. Retrain weights on the new sessions overnight.
- H+40-48** Dry-run a partner-bank walk-through of one tier-3 release and one appeal. Freeze code + PDF.

What is already built and reproducible

- chakravyuh/ -- signal catalogue, mule graph, fusion, FastAPI service (fully typed).
- eval/ -- synthetic-world generator (60k legit + 2.4k scam), 3 seeded worlds, calibration to alert budget.
- web/ -- interactive scenario dashboard (scenario, live scoring, signed receipt).
- tests/ -- 7 pytest smoke tests including 'no single signal can breach tier 2'.

System architecture -- end-to-end

One diagram, four planes: device, bank fraud-ops VPC, UPI rail, data platform.



1. Signals originate on the device (call / SIP state, accessibility scanner, package installer, on-device SMS classifier) and at the bank (payee VPA, first-time flag, amount z-score, staircase, FD-broken).
2. The device SDK gates raw content: only 20 booleans + amount + payee VPA hash reach the network.
3. Bank fraud-ops VPC hosts the stateless decision engine (FastAPI, /decide). It reads only the evidence payload and a mule feature-store row for the payee. It writes only a signed receipt.
4. The UPI PSP hook is used for tier-3 cooling-off holds (issue a debit-freeze token, releasable by the agent desk within 30 min). Tier-2 friction stays in the app.
5. The data platform runs the 7-day payment-graph refresh (mule score), the weekly weight retrain, and the fraud-ops queue for tier-1 receipts. It ingests the I4C suspect registry hourly.

Detection flow, escalation & receipts

One session traced through the engine. The same receipt shows up in every downstream tool.

Step 1 -- session opens

User presses Pay. The SDK collects the 20 booleans; only what is TRUE is sent (efficient + private by default). Payload is signed with the device-attested key so replay is detectable.

Step 2 -- /decide runs

The engine looks up the payee row in the mule feature store (mule_score, age_days, registry_hit) and adds the 3 network signals. It computes stage scores, active_stages, playbook, tier, and top 5 contributing signals.

Step 3 -- action & message

Tier 0 -> allow. Tier 1 -> silent watch. Tier 2 -> in-app interrupt with the specific playbook message and 2 pre-payment questions. Tier 3 -> PSP freeze-token; the UPI PIN dialog does not open. All three carry the same signed receipt so appeal, audit and NCRP filing use one identifier.

Step 4 -- human review

Tier 3 alone reaches the agent desk. Agent sees: (a) the receipt, (b) the playbook message, (c) the top 5 signals, (d) the payee mule profile. Dual-auth release. Agent's release reason is appended to the receipt.

Step 5 -- learning

Confirmed mule payee -> added to the internal suspect list (never publicly exposed). Confirmed FP -> session vector added to the weekly training set. Adversarial adaptations are tracked as a separate cohort so we notice drift within days, not weeks.

Decision receipt (schema)

```
{ log_id, ts_ms, session_id, amount, evidence: {...true only...}, score, tier, playbook, active_stages, action, hmac_sha256(secret, payload) }
```

Why receipts matter

One id ties the SDK, the engine log, the agent desk, and any downstream NCRP / bank-Ombudsman filing. A hold is never a black-box outcome: the user (via app) and any reviewer (via portal) can retrieve the exact evidence and weights that produced it.

Security, privacy, adversarial risk & validation

The rules of the game -- and how we prove we are winning.

Adversarial risks we plan for

Suppressed signals: fraudster tells the victim to hang up before paying. Coverage: adapted-session recall 88% (vs. 92% overall) -- extraction + cash-out stages still fire.

Aged / rented mules: 25% of synthetic mules use accounts >120 days old. Coverage: mule_score stops relying on account age alone; distinct-sender fan-in and passthrough ratio still catch them.

Amount splitting: victim asked to send 4 x 25k instead of 1 x 100k. Coverage: 'staircase' signal + session-level score across a 60-minute window.

Deepfake bypass of video call: does not help -- signal is 'video call is active', not 'face is real'.

Poisoning: rented accounts trying to look legit. Coverage: nightly retrain uses only bank-confirmed labels and mule flags, never user-reported ones alone.

Validation plan (Round-1 evidence)

Synthetic worlds are seeded and reproducible (seeds 7 / 21 / 11).

Every scenario carries a 25% 'adapted' cohort so we report a bounded worst-case recall, not just the average.

Thresholds are calibrated on a validation world to a bank-set alert budget, then reported on an unseen test world.

Compared against 2 deterministic rules and a black-box gradient-boosting baseline at the same budget.

Ablation ready: removing each stage's signals to show how much of the recall each contributes.

In pilot: 8-week shadow mode, then A/B with control cohort, then value-recall reported per week.

Privacy contract (in-app disclosure + DPA)

On-device signals are computed by the SDK and only booleans leave the device. Raw call audio, screen frames, message text, contact list, and app inventory are never sent, stored, or forwarded to any third party -- including Amazon. The bank retains signed receipts for regulatory audit (7 years, RBI norm), separated from PII where allowed. Users can retrieve all receipts against their id via the bank app; they can request deletion of tier-0 receipts after 90 days. This is a DPDP Act 2023 compliant design.

Appendix -- assumptions, disclosures, references

Assumptions (all synthetic-data-only)

- Deployment prevalence assumed for reporting: 1 scam in 5,000 high-value P2P sessions.
- Alert budget (SLO): tier-1 ≤ 20 / 1k, tier-2 ≤ 3 / 1k, tier-3 ≤ 0.5 / 1k legit sessions.
- Test world: 60,000 legit + 2,400 scam sessions per synthetic world, unseen during weight learning and threshold calibration.
- Adapted-fraudster cohort: 25% of scam sessions have call/video/screen-share signals suppressed.

AI-generated content, datasets, models & third-party assets

- All prose in this deck was drafted by the team and refined with Claude Code (assistant); no reviewer-facing chart, table, diagram, or number was generated -- every stat is computed from eval/run_eval.py.
- No real customer, bank, or platform data is used anywhere in this submission. All sessions and payment graphs are synthetic (see chakravyuh/synth.py and chakravyuh/mule_graph.py).
- Third-party libraries: FastAPI, uvicorn, scikit-learn, numpy, networkx, matplotlib, reportlab, svglib -- all permissively licensed. Fonts: DejaVu Sans (public domain).
- Public references used only for stated prevalence context, cited below. No proprietary or leaked data.

References

- Ministry of Home Affairs, reply in Lok Sabha, 22 July 2025: ₹22,845 crore lost to cyber fraud in 2024; 36.37 lakh NCRP+CFCFRMS complaints.
- Reserve Bank of India, Annual Report 2024-25 (May 2025): UPI-related frauds ₹981 crore across 12.64 lakh incidents in FY25; total digital-payment fraud instances 23,953.
- Indian Cyber Crime Coordination Centre (I4C): Suspect Registry launched September 2024; ~11 lakh suspect identifiers, 24 lakh mule accounts flagged.
- Digital Personal Data Protection Act, 2023: privacy contract of this design.